Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Software Design Description (SDD)
Serial Driver Module

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	Thomas Marmeth Software Designer	2011-10-05
Verified :	Eric Malo, Eng. System Engineer	2011-10-05
Approved :	Martin Ouellet, Eng. Software Department Supervisor	2011/10/05

Document No.	**201111582**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-05**

Revision List

Rev.	ECD#	Date	By	Description of changes
1	---	2011-10-05	Thomas Marmeth	Released version.
0.1	---	2011-09-30	Thomas Marmeth	Implemented comments from inspection I-1809
0	---	2011-08-25	Thomas Marmeth	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE Std 1558-2005, IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems approved on 2005-05-23.

Table of Contents

Revision List 2
Preface 3
Table of Contents 4
List of Figures 5
List of Tables 5

1 INTRODUCTION 6

1.1.1 Purpose 6
1.1.2 Scope 6
1.1.3 Definitions and Acronyms 6
1.1.4 Document Organization 7

2 REFERENCES 9

3 SOFTWARE DESIGN 10

3.1 Decomposition Description 10
3.1.1 Module Decomposition 10
3.1.2 Concurrent Process Decomposition 11
3.2 Dependency Description 12
3.2.1 Inter-Module Dependencies 12
3.2.2 Inter-Process Dependencies 12
3.2.3 Data Dependencies 12
3.3 Interface Description 13
3.3.1 Serial Driver Module functions 13
3.4 Detailed Design 15
3.4.1 Serial Driver Module Design 15
3.4.2 Data Entities Detailed Design 21

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 10
Figure 3-2: Inter-Process Dependencies Diagram 11
Figure 3-3: Serial Port Block Diagram 12
Figure 3-4: Serial Driver Encode Packet 17
Figure 3-5: Serial Driver Decode Packet 18
Figure 3-6: Serial Driver Task String Rx 19
Figure 3-7: Serial Driver Isr Rx 20
Figure 3-8: Serial Driver Isr Tx 21

List of Tables

Table 3-1: PEC Channels Initialization 15

1 INTRODUCTION

This document is the Software Design Description (SDD) of the Serial Driver Module developed by Vapor Stone Rail Systems.

The Serial Driver Module can be part of any of the Door Module System (DMS) Applications. The module is only working with ST C167 processor as it is using the Peripheral Event Controller (PEC).

1.1.1 Purpose

The purpose of this Software Design Description is to describe the software components and implementation details of the Serial Driver Module.

The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.1.2 Scope

The Serial Driver Module is a software module of the Door Module System (DMS) Application.

1.1.3 Definitions and Acronyms

1.1.3.1 Definitions

Boot Loader	Software starting the Door Control Module application.
Car Network	Data bus and optionally trainlines connecting the Door Control Module with the rest of the train (others Door Control Modules in the car and others cars).
Communication Gateway	A specific processor based electronic module running a Communication Gateway Module Application. It acts as gateway between the Door Control Module Application and the Car Network/Portable Test Application.
Door Module System	A term that is used for generically identifying a processor based electronic module running any software within any Door Control Unit.
Door Control Unit	The final unit that is used to control one or more door panels. This unit is normally made of one or two door control modules running the Door Control Module Application and possibly one Communication Gateway Module Application.
Door Control Module	A specific processor based electronic module running a Door Control Module Application. It is used for controlling the door motor.
HW_COM	Communication interface used for communication between Portable Test Application and Door Control Module application, that can be USB, serial or Ethernet.

1.1.3.2 Event Level Definitions

Event:	Term used to globally identify an Info, an Exception or a Failure.
Info:	Info is an occurrence of an Informative condition monitored by the system, which is neither an exception nor a failure.
Exception:	An exception is an occurrence of a condition that is outside of the specified parameters and affecting an outside Line Replaceable Unit (LRU).

Failure: A failure is an occurrence of a condition that is outside of the specified parameters and affecting this Line Replaceable Unit. A failure normally requires maintenance or replacement to restore affected unit to its normal operation conditions.

1.1.3.3 Acronyms

ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
Char	Character
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DMS	Door Module System
ECD	Engineering Change Document
EOM	End Of Message
HW	Hardware
IEEE	Institute of Electrical and Electronics Engineers
IRAM	Internal Random Access Memory
KB	Kilobytes
LED	Light Emitting Diode
LRU	Line Repleacable unit
MHz	MegaHertz
ms	millisecond
NVRAM	Non-Volatile Random Access Memory
OS	Operating System
PEC	Peripheral Event Controller
PTA	Portable Test Application
RAM	Random Access Memory
RS232	Recommended Standard 232 (Computer Serial Interface)
RTC	Real-Time Clock
RTS	Request To Send
RX	Receiver
SCI	Software Configuration Items
SDD	Software Design Description
SFR	Special Function Register
SRAM	Static Random Access Memory
SRS	Software Requirements Specifications
ST	Service and Technology
Std	Standard
TS	Technical Specification
TX	Transmitter
VSRS	Vapor Stone Rail Systems

1.1.4 Document Organization

The Software Design Description (SDD) document describes the architectural software design of the DMS Application Serial Driver Module. This SDD is divided into 3 sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software design summarizes followings:
    - Decomposition Description: describes software entities.

- Dependency Description: gives a brief description of the dependency between each entity.
- Interface Description: describes the functions of each entity.
- Detailed Design: details the functions algorythms implemented by each software entity, presents and explains tasks performed by each software entity.

Within the document, diagrams, flowcharts and tables are used to present the Serial Driver Module software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- Figure 3-1 describes the dependencies between different modules.
- Tables contain data structures used by the Serial Driver Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, SePTAmber 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2005	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

S17-07	Software Design Description Template, Rev. 0, 2002-01-15 complemented with requirements from IEEE Std 1558.
CMM-01	Software Engineering Process, Rev.3
CMMI-02	Software Engineering Policy Manual, Rev. 1
201111489	Portable Test Interface Protocol, Interface Control Document (ICD), VSRS, Rev. 01

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Module Decomposition

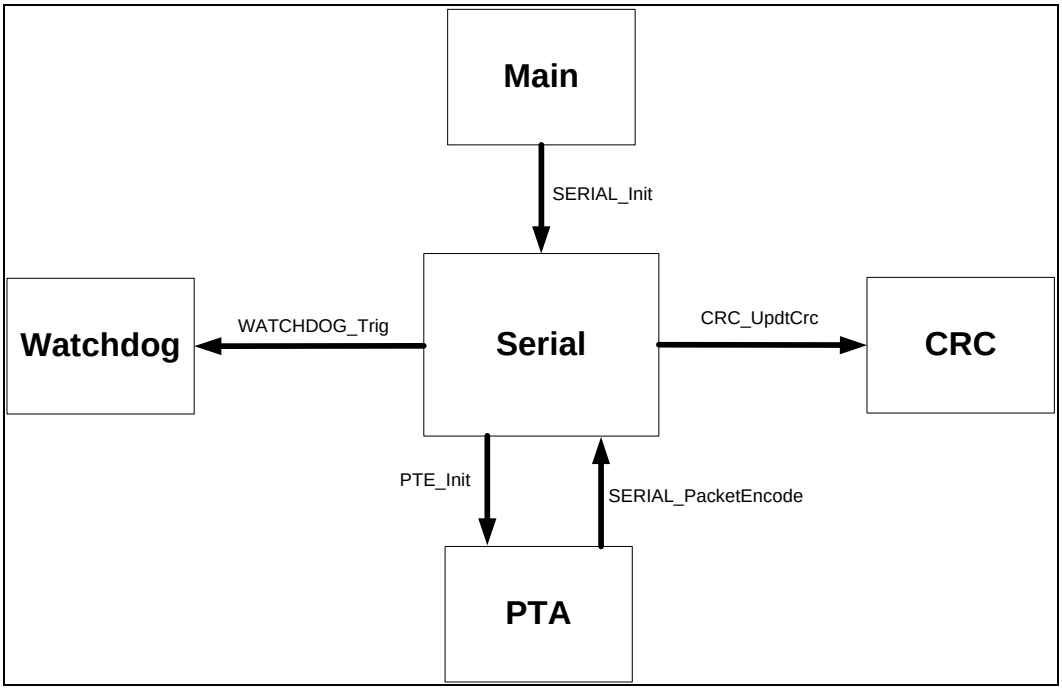

Figure 3-1: Inter-Module Dependencies Diagram

3.1.1.1 Serial Driver Module Description

The Serial Driver Module is a single entity which takes in charge the input/output data of the serial port.

The driver is divided in 3 layers:

- The first layer is a basic service that takes characters from the transmission buffer and sends them over the serial port. For reception, characters are taken from the serial port and copied into a receiving buffer.
- The second layer takes in charge the identification of the type of data received. The only supported protocol is the PTA protocol defined in Interface Control Document #201111489. The data received is copied from the reception buffer to a working buffer. A flag is raised when the message received is completed to inform the decode engine that there is some data received and ready to be decoded. This layer takes care of timeout depending on the type of data received.
- The third layer is the encoder/decoder for the PTA protocol. The message is decoded and redirected to the appropriate service via message queue.

3.1.2 Concurrent Process Decomposition

Figure 3-2 shows the Concurrent process Dependencies Diagram of the Serial Driver Module.

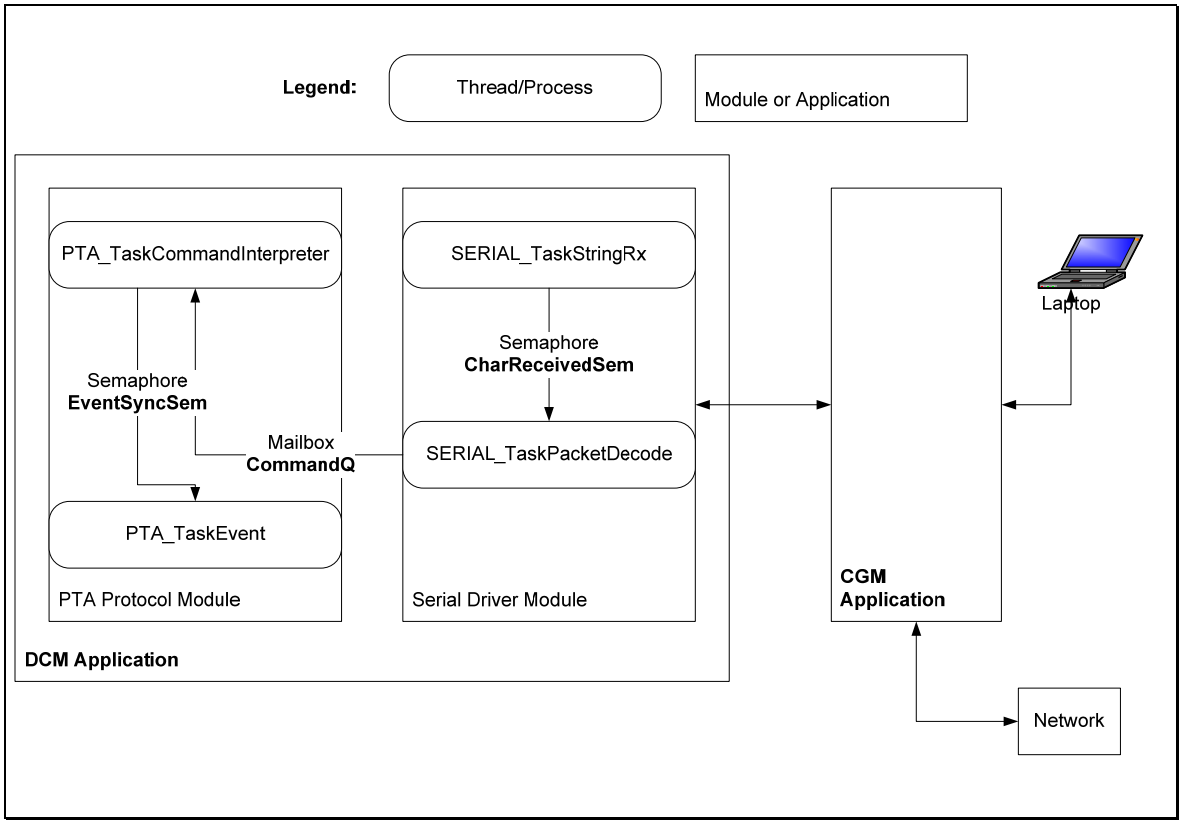

Figure 3-2: Inter-Process Dependencies Diagram

3.1.2.1 Serial Driver Module Task Description

The Serial Driver process is divided in 2 tasks:

- A first task checks if a character is received and present. This task will then validates the content of the message received and send it after validation to PTA_Command Interpreter Task. If the received message contains errors, a message is sent to DmcuToPTAError to notify it to the PTA.
- The second task takes the character present in the reception buffer and copies them into the working buffer.

The module functions shall be designed to support multi-threading.

3.1.2.2 Data Decomposition

4 globals variables are used within the module for data exchanche between the functions :

- ucRxBuf Reception data buffer
- ucRxIbuf reception data ring buffer
- ucTxBuf : transmission data buffer
- WorkBuf.ucData : working buffer

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

3.2.1.1 Serial Driver Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the Serial Driver Module.

3.2.2 Inter-Process Dependencies

Figure 3-3 shows the Inter-Process Dependencies Diagram of the Serial Driver Module.

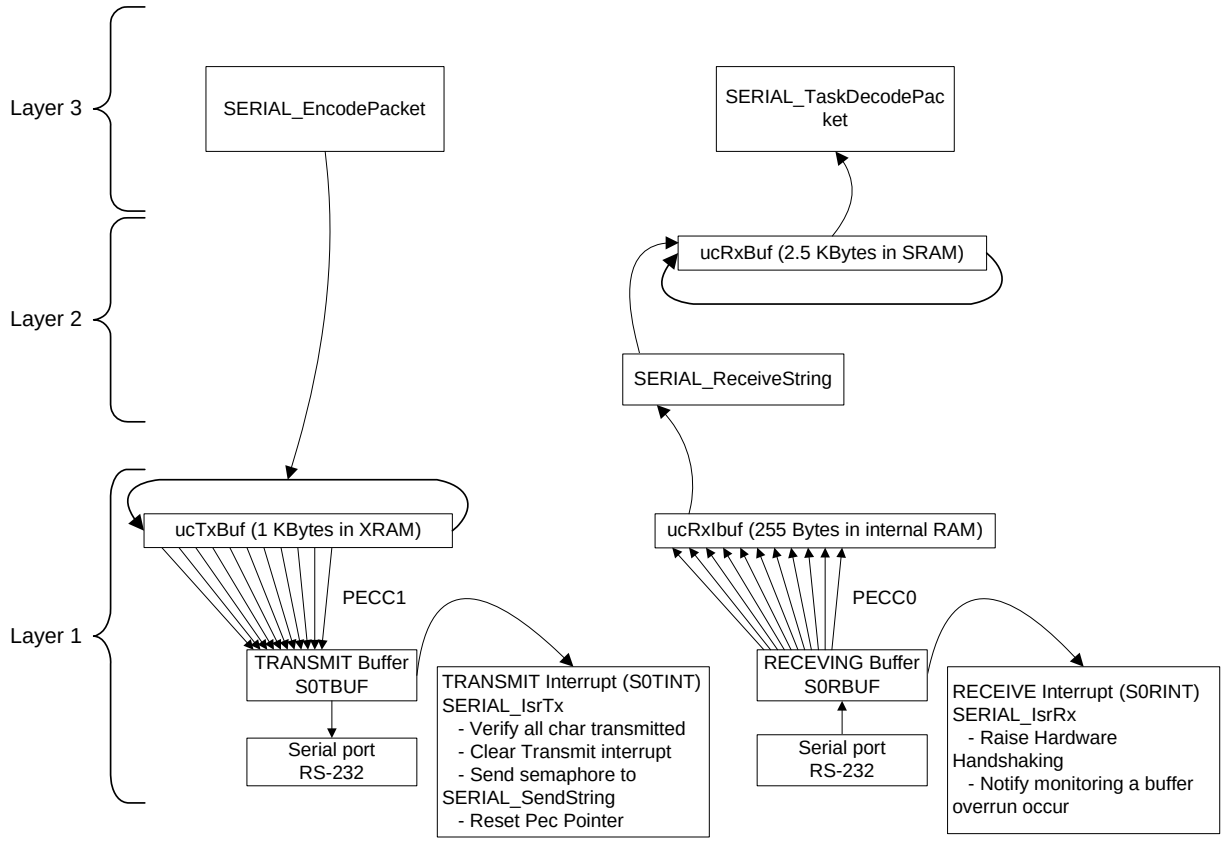

Figure 3-3: Serial Port Block Diagram

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of the Serial Driver Module. The function name, a description of the function, input and output parameters required by the function.

3.3.1 Serial Driver Module functions

The Serial Driver Module implements the following functions (sorted alphabetically):

3.3.1.1 Serial Driver Module External Interfaces

Function Name	bit SERIAL_CharRx (int *piChar)
Description	This function retrieves a char from the ucRxBuf.
Input	int *piChar: Pointer to a char buffer
Return	TRUE if character present in the received buffer FALSE otherwise
Output	Character received in ucRxBuf.

Function Name	bit SERIAL_CharTx (char cChar)
Description	This function takes the received characters and inserts them in transmit buffer and then initiates the transmission if the serial port was idle.
Input	char cChar : Character to send
Return	TRUE if character have been copied successfully in transmit buffer FALSE otherwise
Output	Character in ucTxBuf

Function Name	void SERIAL_Init(int iFcpu, unsigned long ulBaud, unsigned int uiSetting)
Description	This function initializes the Serial component. It effects all necessary configurations of the SFR, depending on the selected operating mode (port settings and baud rate). The configuration determines whether the Serial interrupts are to be released and the priority of the released interrupt.
Input	int iFcpu : Frequency of the CPU board in MHz unsigned long ulBaud : Baud rate of the Serial Port (600, 1200, 2400, 4800, 9600, 19200 ,38400 and 57600) unsigned int uiSetting : Port settings (Data byte number, stop bit, parity, framing overrun, loopback)
Output	None
Return	None

Function Name	bit SERIAL_PacketEncode(unsigned char ucLength, unsigned char xhuge *pucBuf)
Description	This function is used to package the message to be transmitted to the other end. The message is received with its header, length, packet sequence number, command and data if present as per ICD #201111489. The function adds the sequential number and calculates the CRC of the message and adjusts the message length.
Input	unsigned char ucLength : The length of the data. unsigned char xhuge *pucBuf : Pointer to the buffer containing the data to be transmitted
Return	TRUE if string is small enough to fit in buffer FALSE if string is too long
Output	Data to send in ucTxBuf[uiTxStart]

Function Name	void SERIAL_TaskPacketDecode (void)
Description	This function is a separate task and verifies if a char is received and present, after having received a semaphor CharReceivedSem to indicate that characters were available in the receiving buffer ucRxBuf. The task validates the content of the received message by verifying the validity of the command and the CRC of the message and sends it after validation to PTA_Command Interpreter via the working buffer WorkBuf.ucData.
Input	Characters received in ucRxBuf, the receive buffer CharReceivedSem Semaphore indicating that characters have been received
Output	None
Return	Decode message in WorkBuf.ucData

Function Name	void SERIAL_TaskStringRx (void)
Description	This function is a separate task in and takes the chars present in the reception ring buffer ucRxIbuf and copies them into the normal receive buffer ucRxBuf. Pointers of the reception ring buffer ucRxIbuf are then reset to get ready to receive new chars. The task will then release semaphore CharReceivedSem if characters where received
Input	Characters received in ucRxIbuf, the receive ring buffer
Return	None
Output	Characters in ucRxBuf CharReceivedSem Semaphore indicating that characters have been received

3.3.1.2 Serial Driver Module Internal Interfaces

Function Name	void SERIAL_IsrRx(void)
Description	This is the receive interrupt service routine in case of overflow.
Input	None
Return	None
Output	Data received in ucRxBuf

Function Name	void SERIAL_IsrTx(void)
Description	This is the transmit interrupt service routine in case of overflow.
Input	Data to be transmitted in ucTxBuf
Return	None
Output	bTXBuffEmpty TRUE if transmit buffer is empty

3.4 Detailed Design

3.4.1 Serial Driver Module Design

The Serial Port Driver has two different routines to send and receive data from the serial port. Each of them is interrupt driven. The transmission and reception is made automatically with a Peripheral Event Controller (PEC) transfer. Each PEC channel has a source and destination pointer. The source or the destination can be incremented after each transfer. When a PEC channel is active, the PEC takes place of the standard interrupt routine for the number of transfer specified in the COUNT field of the PECCx registers. While the COUNT field has not reached zero, the transfer from the source to the destination occurs in replacement of the interrupt routine. When the COUNT of a PEC channel reaches zero, the standard interrupt is then executed on the next event that triggers the interrupt.
The serial reception is achieved with the PEC channel #0 and the transmission is achieved with the PEC channel #1. Table 3-1 shows the initial configuration for the PECC0 and PECC1. After initialization, the reception of data on the serial port is enabled and the PEC channel 0 starts to transfer characters into the receive ring buffer ucRxIbuf in the internal RAM. A description of the serial port and layers is shown in Figure 3-3. A maximum of 254 bytes can be received without any interruption triggered. When the COUNT value of the PECC0 register reaches zero, the functionality of the PEC is disabled and when the next character is received, the standard interrupt routine associated with the serial port (interrupt #0x2B, S0RINT) is executed.
The task SERIAL_port is executed every 44ms to ensure that the ucRxIbuf is never full. It takes 44.27ms @57600 bauds, 8bits, 1 stop bit, and no parity to fill the 255-byte buffer.

Table 3-1: PEC Channels Initialization for ST processor C167

Name	Value	Description	Attributes
PECC0	0x03FE	Move bytes from the receive buffer (s0rbuf) of serial port to internal ram	Transfer byte, increment destination, maximum of 254 bytes being transferred (0xfe)
SRCP0	S0RBUF (0xFEB2)	Set the source pointer of the pecc0 to the receive buffer (s0rbuf) of the serial port.	
DSTP0	ucRxIbuf	Set the destination pointer of the pecc0 to the designated location in internal ram.	
PECC1	0x0500	Move bytes from the internal ram to the transmit buffer (s0tbuf) of serial port	Transfer byte, increment source.
SRCP1	ucTxIbuf	Set the source pointer of the pecc1 to the designated location in internal ram.	
DSTP1	S0TBUF (0xFEB0)	Set the destination pointer of the pecc1 to the transmit buffer (s0tbuf) of the serial port.	

The interrupt routine for transmission is enabled in the function SERIAL_SendString.

3.4.1.1 SERIAL_CharRx

This function retrieves a char from the ucRxBuf.
This function is too simple to provide an algorithm/flowchart.

3.4.1.2 SERIAL_CharTx

This function takes a received character and insert it in the transmit buffer and then initiate the transmission if the serial port was idle.

This function is too simple to provide an algorithm/flowchart.

3.4.1.3 SERIAL_Init

This function initializes the Serial component. It effects all necessary configurations of the SFR, depending on the selected operating mode (Frequency of the CPU board in MHz, Baud rate of the Serial Port, Port settings: Data byte number, stop bit, parity, framing overrun, loopback). The configuration determines whether the Serial interrupts are to be released, and the priority of the released interrupt.

This function will initialize the variables of the module and set the baud rate. It will then enable baudrate generator and the receiver. The last actions of the function are to create semaphore and mailbox for serial task and command interpreter task, and then to create the Serial task.
This function is too simple to provide an algorithm/flowchart.

3.4.1.4 SERIAL_PacketEncode

This function receives a command and its associated data to be transmitted. The command is received by the PTA Command Interpreter and can be a re-transmission of the last packet command or a complete packet. The function receives the message to be transmitted that contains the header and the data. If there is data associated with it, the function adds the EOM, calculates the CRC and adjusts the message length. The pointer in ucTx_Buf is adjusted and the preparation of the PECC1 begins to start the transmission if the ucTx_Buf was empty and no transmission in progress.
If the received command asks to re-transmit the last message, the pointer in ucTx_Buf is readjusted and the preparation of the PECC1 starts the transmission if the ucTx_Buf was empty and no transmission was in progress.
Figure 3-4 shows the sequence used to process the Serial Driver Encode Packet.

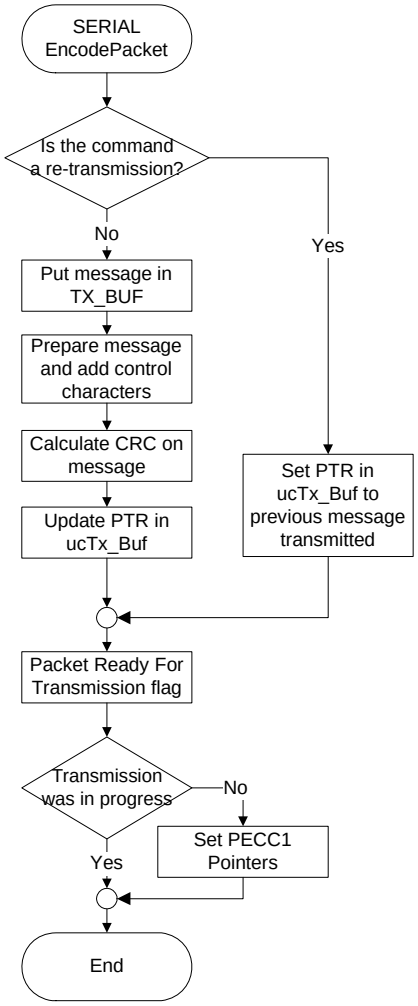

Figure 3-4: Serial Driver Encode Packet

3.4.1.5 SERIAL_TaskDecodePacket

This function completes the Serial Driver Task String Rx. Figure 3-5 shows the sequence used to process the Serial Driver Decode Packet.

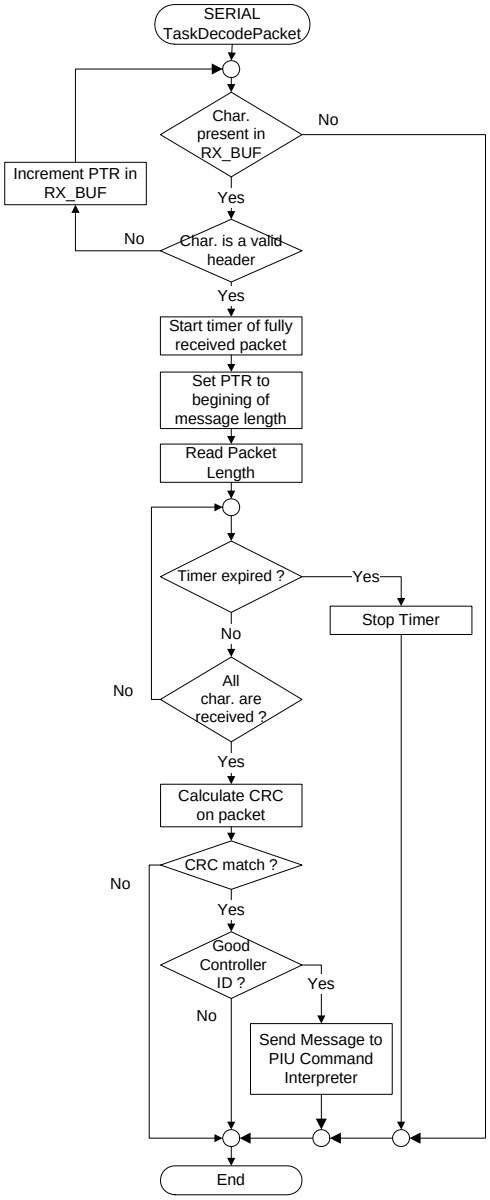

Figure 3-5: Serial Driver Decode Packet

3.4.1.6 SERIAL_TaskStringRx

This task is scheduled with a period of 40ms. It verifies if characters are presents in the ucRxIbuf. When characters are presents, the function copies them from the ucRxIbuf to the ucRx_Buf and then resets the PECC0 counter and destination pointer to the beginning of the ucRxIbuf. In case there is no character present in the ucRxIbuf, the function stops its execution and waits the next occurrence.
Figure 3-6 shows the sequence used to process the Serial Driver Task String Rx.

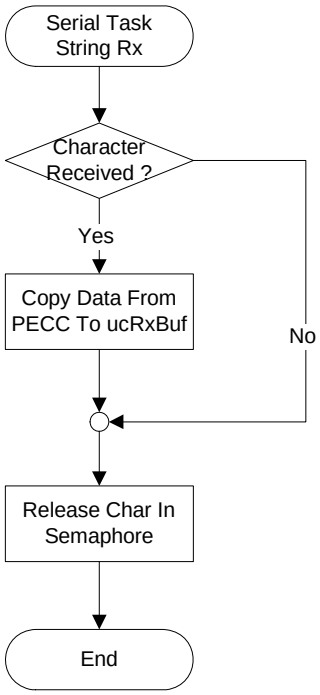

Figure 3-6: Serial Driver Task String Rx

3.4.1.7 SERIAL_IsrRx

This function is used only when an overflow of the ucRxIbuf occurs. The value of the counter is included in PECC0 register and when it goes from 1 to 0, the interrupt is then triggered and the SERIAL_IsrRx is executed. The function set a flag to indicate that a buffer overflow occurred. The flag is then used by the SERIAL_ReceiveString function. The hardware handshaking is then activated to stop the incoming flow of characters until the ucRxIbuf is freed. The interrupt flag is reset on exit.

Figure 3-7 shows the sequence used to process the Serial Driver Isr Rx.

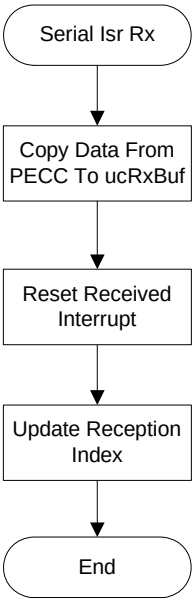

Figure 3-7: Serial Driver Isr Rx

3.4.1.8 SERIAL_IsrTx

The interrupt routine is triggered when the last byte on the serial port is transmitted. This function is used to reset the PECC source pointer when the ucTxIbuf is empty after the last byte is transmitted. The function disables the transmit interrupt and sets a flag to indicate that the transmission is not in progress. If there are more characters to send, the PEC channel 1 is then set and reloaded with new values.

Figure 3-8 shows the sequence used to process the Serial Driver Isr Tx.

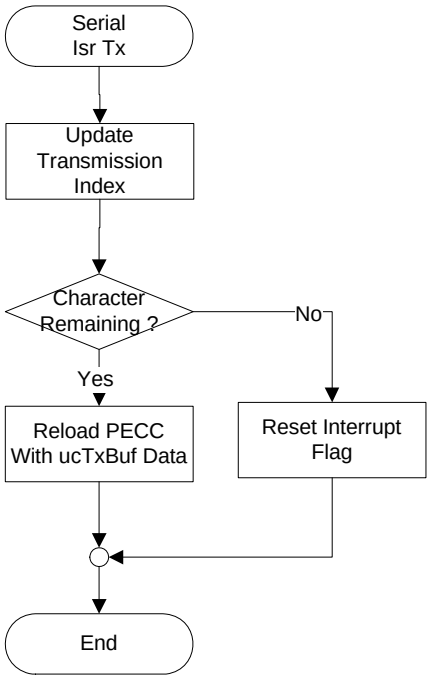

Figure 3-8: Serial Driver Isr Tx

3.4.2 Data Entities Detailed Design

Not applicable.